Resora Allocation System

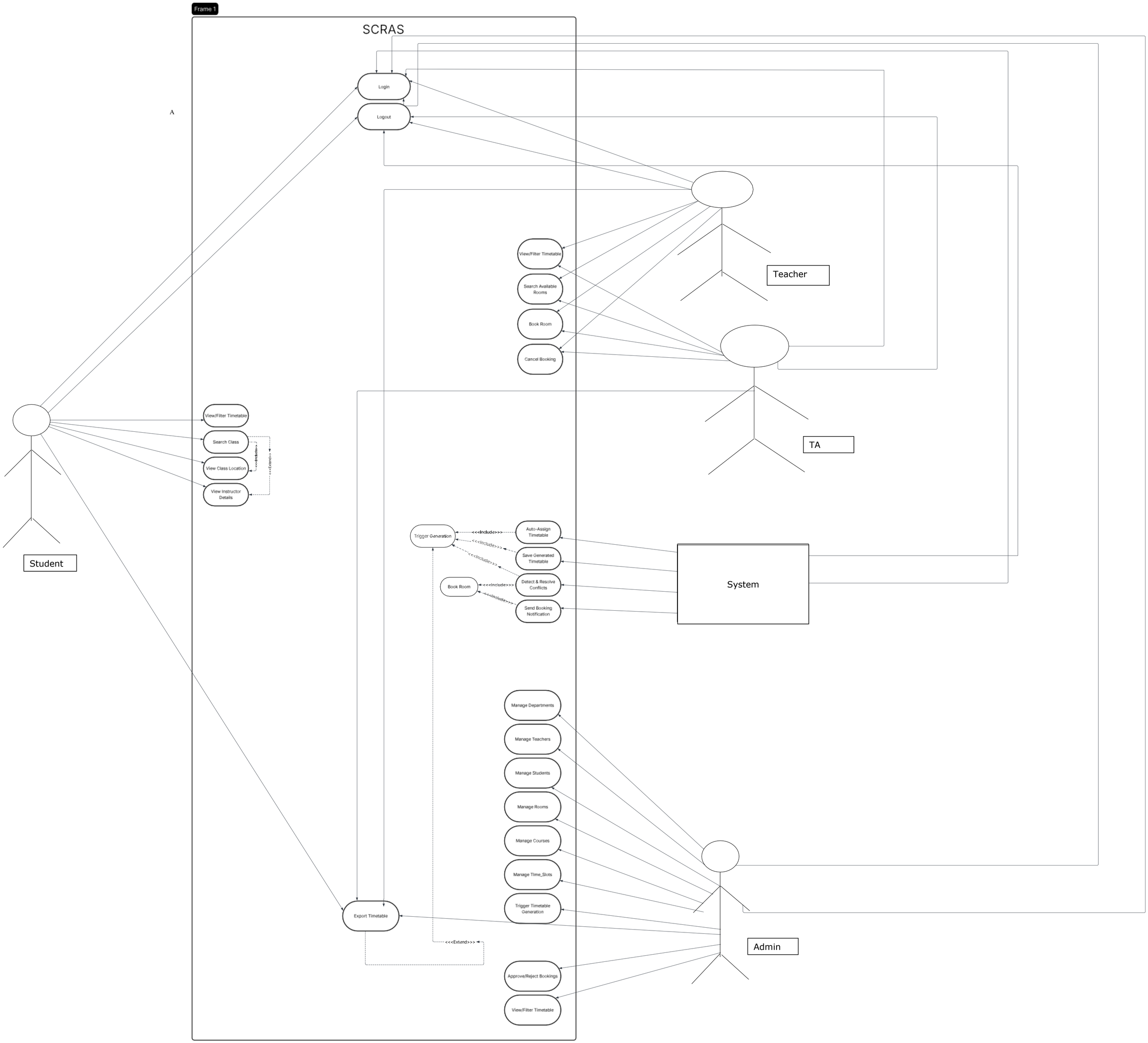
Phase-1

Team # 4

Member Name	Member Roll #	Primary Responsibility
<Team lead>	M. Subhan Noor	Class Analysis diagram
<Member>	Wali Ahmad	Use Case diagram
<Member>	Amna Islam	Use Case Description
<Member>	Mustafa Basit	Use Case Description

Table of Contents

1. Use Case Diagram 1
2. Use Case Descriptions 2
3. Analysis Class Diagram 3-25


Frame 1
SCRAS
A
Login
Logout
View/Filter Timetable
Search Available Rooms
Book Room
Cancel Booking
Teacher
TA
View/Filter Timetable
Search Class
View Class Location
View Instructor Details
<<Include>>
<<Extend>>
Student
Trigger Generation
<<Include>>>
Auto-Assign Timetable
Save Generated Timetable
Detect & Resolve Conflicts
Send Booking Notification
Book Room
System
Manage Departments
Manage Teachers
Manage Students
Manage Rooms
Manage Courses
Manage Time_Slots
Trigger Timetable Generation
Export Timetable
<<Extend>>>
Approve/Reject Bookings
View/Filter Timetable
Admin

Identifier	UC-01
Name	Login
Purpose	Allows users to securely access the system using valid credentials, ensuring authentication before granting access to system features.
Priority	High
Actors	Student, Teacher, TA, Admin
Pre-condition(s)	• User must be registered in the system • User must have valid login credentials (username & password) • System must be operational
Post-condition(s)	• User is successfully authenticated • User entry is created in Active Session table with session_id • User is redirected to their respective dashboard

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User navigates to the login page	System displays the login form with username and password fields
2	User enters username and password	System validates that input fields are not empty and formats are correct
3	User clicks the login button	System checks the provided credentials against the database
4	—	System generates session_id
5	—	System stores session in Active Session table
6	—	System authenticates the user
7	—	System redirects the user to their respective dashboard based on their role

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	User enters incorrect credentials	System displays an error message: "Invalid username or password. Please try again."
2	User leaves required fields empty and clicks login	System highlights the empty fields and prompts the user to fill in the required information
3	User account is blocked	System denies access and displays the account status message with instructions to contact admin
4	User fails login 3 consecutive times	System temporarily locks the account and notifies the user via on-screen message

Identifier	UC-02
Name	Logout
Summary	Allows users to securely terminate their current session and exit the system, preventing unauthorized access to their account.
Priority	High
Actors	Student, Teacher, TA, Admin
Pre-condition(s)	• User must be logged into the system • User must exist in Active Session table
Post-condition(s)	• User session is fully terminated • User is redirected to the login page • User removed from Active Session table

• **Typical Course of Action**		
S#	**Actor Action**	**System Response**
1	User clicks the logout button in the navigation bar	System processes the logout request but before termination the system save the changings of user
2	—	System invalidates
3	—	System clears any cached user data from the browser/client
4	—	System redirects the user to the login page with a success message

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	User's session has already expired before clicking logout	System detects no active session and automatically redirects the user to the login page
2	User attempts to logout without an active session	System displays a message: "No active session found. You are not currently logged in."
3	Network error occurs during logout	System displays an error message and retains the session until a successful logout is confirmed

Identifier	UC-03
Name	Manage Department
Summary	Enables admin to add, update, or delete department records to maintain accurate organizational structure within the system.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Admin must have department management privileges
Post-condition(s)	• Department information is updated in the system • Changes are saved in the database • Affected records are updated accordingly

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Department Management section	System displays the full list of existing departments with options to add, edit, or delete
2	Admin clicks 'Add Department' or selects an existing department to edit/delete	System displays the department form pre-filled with data (for edit) or blank (for add)
3	Admin enters or updates the department name and relevant details	System validates the entered data for completeness and uniqueness
4	Admin clicks Submit to save changes	System saves the new or updated department record to the database
5	—	System displays a confirmation message: "Department saved successfully"

Alternate Course of Action

S#	Actor Action	System Response
1	Admin enters a department name that already exists	System displays a validation error: "Department name already exists. Please use a unique name."
2	Admin attempts to delete a department that is linked to existing courses or teachers	System prevents the deletion and displays a dependency warning listing the linked records
3	Admin submits the form with empty required fields	System highlights the missing fields and prompts the admin to fill in the required information
4	Admin enters invalid characters in the department name	System shows an inline validation error: "Invalid characters detected. Only letters, numbers, and spaces are allowed."

Identifier	UC-04
Name	Manage Teachers
Summary	Enables admin to manage teacher records by adding new teachers, updating existing information, and removing teachers from the system.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Teacher data module must be accessible
Post-condition(s)	• Teacher records are updated in the system • Database reflects the latest changes • Teacher availability is updated for scheduling

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Teacher Management section	System displays the list of all registered teachers with search and filter options
2	Admin selects Add, Edit, or Delete for a teacher	System displays the appropriate form or confirmation dialog
3	Admin enters or updates teacher details (name, email, department, availability)	System validates all input fields for correct format and completeness
4	Admin confirms the action by clicking Submit	System updates the teacher record in the database
5	—	System displays a success confirmation message and refreshes the teacher list

Alternate Course of Action

S#	Actor Action	System Response
1	Admin enters a duplicate teacher record (same name and email)	System detects the duplication and shows an error: "A teacher with this information already exists."
2	Admin inputs data in an invalid format (e.g., invalid email)	System highlights the invalid field and displays the appropriate validation message
3	Admin attempts to delete a teacher who is currently assigned to active courses	System restricts the deletion and displays a warning: "This teacher is assigned to active courses. Reassign before deleting."
4	Admin searches for a teacher that does not exist	System displays a message: "No records found matching your search criteria."

Identifier	UC-05
Name	Manage Students
Summary	Allows admin to manage student records, including enrolling new students, updating their profiles, and removing students from the system.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Student data module must be accessible
Post-condition(s)	• Student records are updated • Database reflects current enrollment data

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin navigates to the Student Management section	System displays the list of all enrolled students with search and filter options
2	Admin selects Add, Edit, or Delete for a student	System displays the appropriate form or confirmation dialog
3	Admin enters or updates student details (name, ID, department, enrolled courses)	System validates all input fields for completeness and correct format
4	Admin clicks Submit to confirm the action	System saves or updates the student record in the database
5	—	System shows a success message and refreshes the student list

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Admin enters a duplicate student ID	System displays an error: "A student with this ID already exists in the system."
2	Admin inputs data in invalid format	System highlights the invalid field and shows the relevant validation message
3	Admin tries to delete a student currently enrolled in active courses	System prevents deletion and shows: "Student is enrolled in active courses. Unenroll first before deleting."
4	Admin searches for a student that does not exist	System displays: "No student records found matching your search."

Identifier	UC-06
Name	Manage Rooms
Summary	Enables admin to manage room records including adding new rooms, updating room details (capacity, type, facilities), and removing rooms from the system.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Room management module must be accessible
Post-condition(s)	• Room records are updated in the system • Room availability is accurately reflected for scheduling and booking

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin navigates to the Room Management section	System displays a list of all available rooms with their details and current status
2	Admin selects Add, Edit, or Delete for a room	System presents the relevant form pre-filled with room data (for edit) or blank (for add)
3	Admin enters or updates room details (room number, building, capacity, type, facilities)	System validates the input to ensure all required fields are filled and values are valid
4	Admin clicks Submit to save changes	System updates the room record in the database
5	—	System displays a confirmation: "Room information saved successfully" and refreshes the room list

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Admin enters a room number that already exists	System shows a duplication error: "This room number is already registered. Please use a unique identifier."
2	Admin attempts to delete a room that is currently booked or assigned to the timetable	System blocks the deletion and displays: "Room is currently in use. Remove existing bookings/assignments before deleting."
3	Admin enters invalid capacity value (e.g., negative number or text)	System displays a validation error: "Please enter a valid positive integer for room capacity."
4	Admin submits the form with missing required fields	System highlights the empty fields and prompts: "Please fill in all required fields before submitting."

Identifier	UC-07
Name	Manage Courses
Summary	Allows admin to manage course records, including creating new courses, updating course details, and removing courses from the system.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Course management module must be accessible • Departments must exist in the system
Post-condition(s)	• Course records are updated • Scheduling data is adjusted based on course changes

• Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Course Management section	System displays all existing courses with options to search, add, edit, or delete
2	Admin selects Add, Edit, or Delete for a course	System shows the relevant form or confirmation dialog
3	Admin enters or updates course details (code, title, department, credit hours, assigned teacher)	System validates all required fields and checks for conflicts
4	Admin clicks Submit	System saves the course record and updates scheduling data accordingly
5	—	System displays a confirmation: "Course saved successfully" and refreshes the course list

Alternate Course of Action

S#	Actor Action	System Response
1	Admin enters a duplicate course code	System shows an error: "A course with this code already exists. Please use a unique course code."
2	Admin tries to delete a course that is included in an existing timetable	System prevents deletion and displays: "This course is part of an active timetable. Remove it from the schedule first."
3	Admin assigns a teacher who is unavailable or already overloaded	System flags the conflict and displays: "Selected teacher has a scheduling conflict. Please choose an available instructor."
4	Admin submits with missing required fields	System highlights empty fields and prompts: "All required fields must be completed before saving."

Identifier	UC-08
Name	Manage Time Slots
Summary	Allows admin to define, update, and remove time slots used by the scheduling engine to assign classes to appropriate periods.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • Time slot management module must be accessible
Post-condition(s)	• Time slot data is updated in the system • Scheduling engine uses updated time slots for timetable generation

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Time Slot Management section	System displays all configured time slots with start time, end time, and day assignments
2	Admin selects Add, Edit, or Delete for a time slot	System presents the relevant form or confirmation dialog
3	Admin enters or updates time slot details (day, start time, end time, label)	System validates the data and checks for overlapping time slots
4	Admin clicks Submit	System saves the time slot configuration to the database
5	—	System shows a confirmation: "Time slot saved successfully" and refreshes the list

Alternate Course of Action

S#	Actor Action	System Response
1	Admin enters a time slot that overlaps with an existing one	System detects the conflict and displays: "This time slot overlaps with an existing slot. Please adjust the timing."
2	Admin tries to delete a time slot that is currently assigned in the timetable	System prevents the deletion and shows: "Time slot is in use by the current timetable. Remove it from the schedule first."
3	Admin enters end time before start time	System shows a validation error: "End time must be later than start time. Please correct the values."
4	Admin submits with missing fields	System highlights the empty fields and prompts: "Please complete all required fields before saving."

Identifier	UC-09
Name	Trigger Timetable Generation
Summary	Allows admin to initiate automatic timetable generation based on available rooms, teachers, courses, and time slots, using the scheduling engine.
Priority	High
Actors	Admin
Pre-condition(s)	• Admin must be logged in • All required data (rooms, teachers, courses, time slots) must exist and be configured in the system
Post-condition(s)	• Timetable generation process is initiated and completed • A draft timetable is created and available for review

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Timetable Generation section	System displays the generation options and current data summary
2	Admin reviews data completeness and clicks "Generate Timetable"	System collects all required data (rooms, courses, teachers, time slots)
3	—	System initiates the scheduling algorithm and begins processing
4	—	System automatically assigns resources based on constraints and availability
5	—	System displays the generated timetable for review and confirmation

Alternate Course of Action

S#	Actor Action	System Response
1	Required data is missing (e.g., no rooms or time slots configured)	System stops the process and shows: "Incomplete data. Please ensure all rooms, courses, teachers, and time slots are configured."
2	A scheduling conflict is detected during generation	System triggers the conflict resolution process and highlights the conflicting entries for admin review
3	System fails to generate a complete timetable due to over-constraints	System displays a failure message: "Unable to generate a complete timetable. Please review constraints and try again."
4	Admin triggers generation while a previous generation is still in progress	System displays: "A generation process is already running. Please wait for it to complete."

Identifier	UC-10
Name	Auto Assign Schedule
Summary	Enables the system to automatically assign courses to rooms and time slots based on teacher availability, room capacity, and scheduling constraints.
Priority	High
Actors	Admin, System
Pre-condition(s)	• Admin must be logged in • Timetable generation must have been triggered • All data prerequisites (courses, rooms, teachers, time slots) must be available
Post-condition(s)	• Schedule assignments are saved • Timetable entries are created for all assigned courses • Conflicts are flagged for review

Typical Course of Action

S#	Actor Action	System Response
1	System receives trigger from the Timetable Generation process	System loads all unassigned courses, available rooms, teachers, and time slots
2	—	System evaluates teacher availability and room capacity constraints
3	—	System matches each course to a compatible room and time slot
4	—	System saves all successful assignments to the schedule database
5	Admin reviews the auto-assigned schedule	System displays the complete schedule with all assignments and any flagged conflicts

Alternate Course of Action

S#	Actor Action	System Response
1	No available room fits the course requirements at any time slot	System flags the course as unassigned and notifies the admin: "No suitable room available for [Course Name]. Manual assignment required."
2	Teacher has conflicting assignments for two courses at the same time slot	System detects the conflict, skips one assignment, and flags it for admin to resolve manually
3	All time slots are occupied and courses remain unassigned	System generates a partial schedule and displays: "Some courses could not be assigned due to full schedule. Please add more time slots or rooms."
4	Admin cancels the auto-assign process mid-execution	System halts processing, discards partial assignments, and restores the previous schedule state

Identifier	UC-11
Name	Detect & Resolve Conflict
Summary	Allows the system and admin to identify scheduling conflicts (e.g., double-booked rooms or teachers) and apply resolutions to produce a valid, conflict-free timetable.
Priority	High
Actors	Admin, System
Pre-condition(s)	• Admin must be logged in • A timetable or schedule must exist • Conflicts must be present in the current schedule
Post-condition(s)	• Detected conflicts are resolved • Updated schedule is saved • Conflict log is maintained for audit purposes

Typical Course of Action

S#	Actor Action	System Response
1	System scans the generated timetable for conflicts after generation or on demand	System identifies all scheduling conflicts (same room, same teacher, overlapping time slots)
2	System presents the conflict report to the admin	System displays a detailed list of conflicts with affected courses, rooms, teachers, and time slots
3	Admin reviews each conflict and selects a resolution (reassign room/time/teacher)	System applies the proposed resolution and checks if it introduces new conflicts
4	Admin confirms the resolution	System saves the updated schedule entry and marks the conflict as resolved
5	—	System refreshes the timetable view and shows the updated conflict-free schedule

Alternate Course of Action

S#	Actor Action	System Response
1	No conflicts are detected after the scan	System displays: "No scheduling conflicts found. The timetable is valid."
2	Admin's proposed resolution creates a new conflict	System detects the cascading conflict and alerts the admin: "Proposed resolution introduces a new conflict. Please choose a different option."
3	No alternative room or time slot is available for resolution	System notifies the admin: "No available alternatives found. Please add more resources (rooms/time slots) to resolve this conflict."
4	Admin skips resolving a conflict	System marks the conflict as 'Pending' and keeps it visible in the conflict dashboard for future resolution

UC-11 — Detect & Resolve Conflicts

Identifier	UC-11
Name	Detect & Resolve Conflicts
Summary	System checks for room clashes, teacher double-bookings, and student section conflicts during timetable generation and room booking.
Priority	High
Actors	System (automated — triggered by UC-09 and UC-16)
Pre-condition(s)	Called during timetable generation (UC-09) or during room booking (UC-16).
Post-condition(s)	No schedule or booking is saved with any conflict.

Typical Course of Action

S#	Actor Action	System Response
1		Checks Room Clash: same room_id + same slot_id already exists in Schedules or Bookings.
2		Checks Teacher Clash: same teacher_id + same slot_id already assigned.
3		Checks Student Conflict: same course_code + section + batch_year + slot_id already exists.
4		No conflicts detected — returns success to calling use case.

Alternate Course of Action (S1–S3. Conflict detected)

S#	Actor Action	System Response
1		Conflict detected during generation — system automatically tries a different slot, room, or teacher.
2		Conflict detected during booking — system blocks the booking.
3		Displays available alternative rooms/slots to the user.

UC-12 — Save Generated Timetable

Identifier	UC-12
Name	Save Generated Timetable
Summary	The system saves the timetable to the database. When the scheduling algorithm finishes deciding which course goes in which room, with which teacher, at which time the system takes all of those decisions and stores them permanently in the Schedules table so they aren't lost and everyone can view them.
Priority	High
Actors	System (automated — triggered by UC-09)
Pre-condition(s)	UC-10 has completed successfully with no unresolved conflicts.
Post-condition(s)	All assignments are stored in the Schedules table and visible to all users.

Typical Course of Action

S#	Actor Action	System Response
1		Collects all course room teacher slot assignments from UC-10.
2		Writes each assignment as a record in the Schedules table.
3		Commits the database transaction.
4		Marks the timetable as active and makes it visible to all actors.

Alternate Course of Action (S3. Database write failure)

S#	Actor Action	System Response
3		Database write fails (e.g. connection error).
4		Rolls back the transaction.
5		Retains the previous timetable and notifies Admin of the failure.

UC-13 — View / Filter Timetable

Identifier	UC-13
Name	View / Filter Timetable
Summary	All actors can view the generated timetable and filter it by department, section, batch year, teacher, or room.
Priority	High
Actors	Admin, Teacher, TA, Student
Pre-condition(s)	• A timetable has been generated and saved. • User is logged in.
Post-condition(s)	User sees the filtered timetable relevant to their role.

Typical Course of Action

S#	Actor Action	System Response
1	User navigates to the Timetable View page.	
2		Displays the full timetable as a weekly grid.
3	User applies filters: Department, Section, Batch Year, Teacher, and/or Room.	
4		Queries the Schedules table with the applied filters.
5		Displays the filtered timetable showing: Course Name, Room Number, Building/Floor, Teacher Name, and Time.

Alternate Course of Action (S4. No results match the filter)

S#	Actor Action	System Response
4		No schedule entries match the selected filters.
5		Displays message: 'No timetable found for the selected criteria.'
6	User adjusts the filter values and retries.	

UC-14 — Export Timetable (PDF / Excel)

Identifier	UC-14
Name	Export Timetable (PDF / Excel)
Summary	Admin can optionally export the generated timetable as a PDF or Excel file after generation.
Priority	Medium
Actors	Admin
Pre-condition(s)	• A timetable has been generated. • Admin is logged in.
Post-condition(s)	A formatted timetable file is downloaded by Admin.

Typical Course of Action

S#	Actor Action	System Response
1	Admin views the generated timetable (UC-09 / UC-13).	
2	Admin clicks the Export button.	
3		Displays export format options: PDF or Excel (XLSX).
4	Admin selects desired format.	
5		Generates the file with the current filtered timetable view.
6		Initiates file download to Admin's device.

Alternate Course of Action (S5. File generation fails)

S#	Actor Action	System Response
5		File generation fails (server error).
6		Displays error: 'Export failed. Please try again.'
7	Admin retries the export.	

UC-15 — Search Available Rooms

Identifier	UC-15
Name	Search Available Rooms
Summary	Teacher or TA searches for rooms available for a specific purpose, date, and time slot.
Priority	High
Actors	Teacher, TA
Pre-condition(s)	• User is logged in. • Rooms and time slots exist in the system.
Post-condition(s)	User sees a list of available rooms matching their search criteria.

Typical Course of Action

S#	Actor Action	System Response
1	User navigates to the Room Availability Finder page.	
2		Displays search filters.
3	User selects: Room Type, Date, and Time Slot.	
4	User clicks Search.	
5		Queries Rooms table and cross-references Schedules and Bookings to find free rooms.
6		Displays available rooms as cards showing: Room Number, Building, Floor, Capacity, and Room Type.

Alternate Course of Action (S5. No rooms available)

S#	Actor Action	System Response
5		No rooms are free for the selected criteria.
6		Displays message: 'No rooms available. Try a different time slot or date.'
7	User changes the time slot or date and retries.	

UC-16 — Book Room

Identifier	UC-16
Name	Book Room
Summary	Teacher or TA selects an available room and submits a booking request that requires Admin approval.
Priority	High
Actors	Teacher, TA
Pre-condition(s)	• User is logged in. Rooms and time slots exist. • Timetable has been generated.
Post-condition(s)	Booking record is created in the Bookings table with status = Pending.

Typical Course of Action

S#	Actor Action	System Response
1	User performs room search (UC-15) and views available rooms.	
2	User clicks Book Now on a preferred room.	
3		Triggers UC-11 (Detect & Resolve Conflicts) verifies no room clash for that room + slot + date.
4		No conflict detected creates booking record with status = Pending.
5		Triggers UC-17 (Send Booking Notification) emails Admin for approval.
6		Displays confirmation: 'Booking request submitted. Awaiting Admin approval.'

Alternate Course of Action (S3. Room clash detected)

S#	Actor Action	System Response
3		Conflict detected room already booked for the selected slot.
4		Displays error: 'Room unavailable. Please choose another room.'
5	User selects a different available room.	
6		(3) <continue from S# 3>

UC-17 — Send Booking Notification

Identifier	UC-17
Name	Send Booking Notification
Summary	System automatically emails Admin when a new booking request is submitted, and notifies Teacher/TA of Admin's approval or rejection.
Priority	Medium
Actors	System (automated triggered by UC-16 and UC-18)
Pre-condition(s)	A booking request has been submitted (UC-16) or a decision made (UC-18).
Post-condition(s)	• Admin receives notification of pending booking. • Teacher/TA receives decision notification.

Typical Course of Action

S#	Actor Action	System Response
1		Retrieves booking details: requester name, room number, date, time slot, purpose.
2		Generates notification email/in-app alert for Admin.
3		Sends notification to Admin.
4		Logs the notification event.

Alternate Course of Action (S3. Notification delivery fails)

S#	Actor Action	System Response
3		Email/notification service fails.
4		System retries delivery and logs the failure.
5		Admin can still view pending bookings in the dashboard manually.

UC-18 — Approve / Reject Booking

Identifier	UC-18
Name	Approve / Reject Booking
Summary	Admin reviews pending room booking requests and approves or rejects them. Teacher/TA is notified of the decision.
Priority	High
Actors	Admin
Pre-condition(s)	A booking with status = Pending exists in the system.
Post-condition(s)	Booking status is updated to Approved or Rejected. Teacher/TA is notified.

Typical Course of Action

S#	Actor Action	System Response
1	Admin navigates to the Bookings Management page.	
2		Lists all pending booking requests with: requester name, room, date, time slot, and purpose.
3	Admin reviews a booking request.	
4	Admin clicks Approve or Reject.	
5		Updates booking status to Approved or Rejected.
6		Triggers UC-17 sends email/alert to Teacher/TA with Admin's decision.

Alternate Course of Action (S5. Booking already cancelled by Teacher/TA)

S#	Actor Action	System Response
5		Detects that the booking was already cancelled (UC-19).
6		Displays message: 'Booking no longer active — it was cancelled by the requester.'

UC-19 — Cancel Booking

Identifier	UC-19
Name	Cancel Booking
Summary	Teacher or TA can cancel their own pending room booking before Admin processes it.
Priority	Medium
Actors	Teacher, TA
Pre-condition(s)	• User has a booking with status = Pending. • User is logged in.
Post-condition(s)	Booking status is set to Cancelled. The room and time slot are freed.

Typical Course of Action

S#	Actor Action	System Response
1	User navigates to My Bookings page.	
2		Displays user's bookings with current statuses.
3	User selects a Pending booking and clicks Cancel.	
4		Displays confirmation prompt: 'Are you sure you want to cancel this booking?'
5	User confirms cancellation.	
6		Updates booking status to Cancelled.
7		Frees the room and time slot for other bookings.

Alternate Course of Action (S5. User cancels the cancellation)

S#	Actor Action	System Response
5	User clicks No on the confirmation prompt.	
6		Dismisses the prompt booking remains Pending.

UC-20 — Search Class

Identifier	UC-20
Name	Search Class
Summary	Student searches for their scheduled class using course code, section, and batch year.
Priority	High
Actors	Student
Pre-condition(s)	• Student is logged in. • Timetable has been generated by Admin.
Post-condition(s)	Student sees the exact location and timing of their class.

Typical Course of Action

S#	Actor Action	System Response
1	Student navigates to the Class Locator page.	
2		Displays the search form.
3	Student enters: Course Code or Name, Section, and Batch Year.	
4	Student clicks Search.	
5		Queries Schedules JOIN Rooms JOIN Teachers JOIN TimeSlots.
6		Triggers UC-21 (View Class Location) displays room, building, floor, day, and time.
7		Shows current class status: Upcoming / In Progress / Completed.

Alternate Course of Action (S5. No class found)

S#	Actor Action	System Response
5		No schedule entry matches the entered criteria.
6		Displays message: 'No class found. Please verify course code, section, and batch year.'
7	Student corrects the input and retries.	

UC-21 — View Class Location

Identifier	UC-21
Name	View Class Location
Summary	System displays the physical room details and timing for a student's class, triggered automatically by UC-20.
Priority	High
Actors	Student (result displayed automatically from UC-20)
Pre-condition(s)	UC-20 has returned a valid result.
Post-condition(s)	Student can physically locate their class with room and timing information.

Typical Course of Action

S#	Actor Action	System Response
1		Retrieves room and schedule details from Schedules and Rooms tables.
2		Displays: Room Number, Building Name, Floor Number, Day, Start Time, and End Time.
3		Highlights class status: Upcoming (>30 min away), In Progress, or Completed.
4		Optionally shows a campus building hint or map reference.

Alternate Course of Action (S2. Room data missing from schedule)

S#	Actor Action	System Response
2		Room details are missing from the schedule record.
3		Displays: 'Room: TBA contact your department for location.'

UC-22 — View Instructor Details

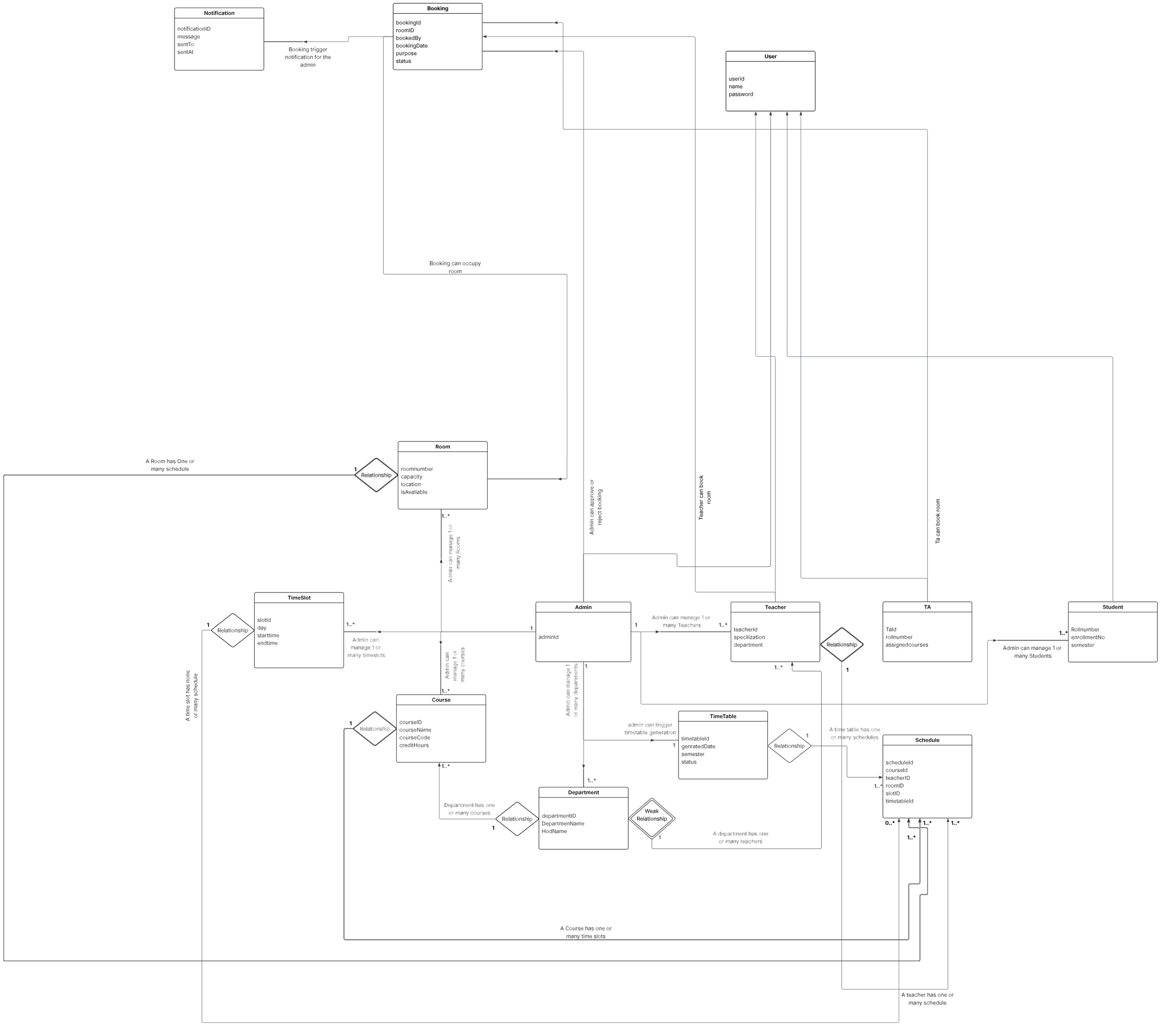
Identifier	UC-22
Name	View Instructor Details
Summary	Student can optionally view teacher contact information and office hours after searching for a class.
Priority	Low
Actors	Student
Pre-condition(s)	UC-20 (Search Class) has returned a result.
Post-condition(s)	Student has access to instructor name, email, and office hours.

Typical Course of Action

S#	Actor Action	System Response
1	Student clicks View Instructor Details on the class search result.	
2		Queries the Teachers table using the teacher_id from the schedule record.
3		Displays: Teacher Full Name, Department, Email Address, and Office Hours (if entered).

Alternate Course of Action (S2. Instructor details incomplete)

S#	Actor Action	System Response
2		Some instructor fields are missing (e.g. office hours not entered).
3		Displays available fields and marks missing ones as 'Not provided.'


Notification
notificationID
message
sentTo
sentAt
Booking trigger notification for the admin
Booking
bookingId
roomID
bookedBy
bookingDate
purpose
status
User
userid
name
password
Booking can occupy room
Room
roomnumber
capacity
location
isAvaliable
A Room has One or many schedule
Relationship
1
1..*
Admin can manage 1 or many Rooms
Admin can approve or reject booking
Teacher can book room
Ta can book room
TimeSlot
slotId
day
starttime
endtime
Admin can manage 1 or many timeslots
Admin can manage 1 or many courses
Admin
adminId
Admin can manage 1 or many Teachers
Teacher
teacherId
specilization
department
TA
TaId
rollnumber
assignedcourses
Student
RollnumberenrollmentNo
semester
Admin can manage 1 or many Students
A time slot has none or many schedule
Course
courseID
courseName
courseCode
creditHours
Admin can manage 1 or many departments
admin can trigger timetable generation
TimeTable
timetableId
genratedDate
semester
status
A time table has one or many schedules
Schedule
scheduleId
courseId
teacherID
roomID
slotID
timetableId
0..*
Department has one or many courses
Department
departmentID
DepartmenName
HodName
Weak Relationship
A department has one or many teachers
A Course has one or many time slots
A teacher has one or many schedule